

Yantra: A Reproducible Recipe for Sub-1B Agentic Tool Calling with Decoupled Routing

Gautam Kishore Eulogik

<https://eulogik.com>

github.com/eulogik · huggingface.co/eulogik

September 16, 2026

Abstract

Small language models have a hard time with agentic tool calling. When a single forward pass has to pick a tool, fill its arguments, and stop cleanly – especially when served through a generic stack that doesn’t match the model’s native runtime – things break fast. We present Yantra, a fully reproducible recipe that turns a 1 B-parameter model into a reliable single-call tool user on a free Colab T4, no paid API calls needed. The core idea is Decoupled Tool Selection / Argument Generation (DTSA): we move tool choice into a cheap, testable runtime router (IDF + char-3-gram + embedding blend, 247/300 routing accuracy) and train the model to emit only the arguments inside a strict bind/args/action-end contract enforced by fixed stop sequences. On a deterministic 300-case ToolACE slice, Yantra reaches a Primary Aggregated Score (PAS) of **0.6775** versus **0.1922** for an identically served reference agentic model (3.52×), with 100% parseable output and 65% exact-argument accuracy. We release the Q4_K_M weights (688 MB, MIT), the training and evaluation pipeline, and a documented negative result: small-data contrastive training (66 pairs) collapses argument quality, while supervised replay on the same fixes recovers safely. Scope is deliberately narrow (single-call, English, $n = 300$); we state every threat to validity inline so the work can be built on without surprises.

Keywords: tool calling, small language models, QLoRA, DTSA, decoupled routing, agentic AI, MiniCPM, reproducibility

Artifacts: Code: <https://github.com/eulogik/yantra> Weights: <https://huggingface.co/eulogik/yantra-1b-agent> License: MIT (code and Yantra weights; base model Apache-2.0).

1 Introduction

Function calling – having a language model invoke external tools and APIs – is the foundation of language agents, enterprise workflows, and on-device assistants [8, 6, 7]. Frontier models do this well, but the devices where agents would be most useful (phones, laptops, edge hardware) can’t run them. Small language models (SLMs) are the natural answer [2], yet a 1 B model asked to read a tool list, pick the right tool, fill every argument correctly, and stop generating – all in one shot – fails in mundane, system-breaking ways: it echoes the tool schema back, mangles argument tags, invents follow-up turns, or never emits a stop token. The failure is sharpest under *generic serving*: a model trained for one runtime (e.g., SGLang with a native parser) degrades badly when served through another (e.g., llama.cpp), which is exactly the situation an open, reproducible project faces.

This paper asks a practical question: with one free Colab T4 GPU, zero paid API calls, and a 1 B base model, how far can careful systems engineering plus lightweight fine-tuning move single-call tool accuracy? Our answer is Yantra, a recipe rather than just a checkpoint:

1. **DTSA (Decoupled Tool Selection / Argument Generation).** Tool choice is removed from the language model entirely. A runtime router binds `<bind tool="...">`; the model generates only the `<args>` block closed by `<action_end/>`. Routing accuracy becomes an independent, GPU-free engineering target.
2. **Router V2 + embedding blend.** An IDF-weighted lexical scorer with char-3-gram name matching (244/300), blended 50/50 with MiniLM cosine similarity (247/300), with per-query normalization. No retraining required to upgrade routing.
3. **A strict generation contract.** Fixed stop sequences and a last-bind-with-args parser (with a tagless fallback) take stopped cleanly from 0.12 to 1.00.
4. **A metric and protocol that fit the claim.** PAS, the mean of eight sub-metrics, on a deterministic 300-case ToolACE slice (seed 42), with temperature 0 and a fully scripted harness.
5. **Full reproducibility and honest negatives.** Every stage reruns on a free T4; weights, code, and notebooks are public. A collapsed DPO round (66 pairs) is reported with numbers, not buried.

On ToolACE-300, Yantra scores PAS **0.6775** versus **0.1922** for the reference agentic 1 B model served identically (3.52 $\times$), is parseable on 100% of cases, and gets arguments exactly right 65% of the time. We do *not* claim state of the art against 7 B+ models, multi-turn planning, or external benchmarks we did not run (notably BFCL); Section 7 lists every limitation we know. The intended contribution is a solid, reusable systems recipe at the 1 B scale, and evidence about which cheap interventions actually move the needle.

2 Related Work

Tool-use language models. Toolformer [11] showed LMs can teach themselves API use via self-supervised data; ReAct [10] interleaves reasoning traces with actions for multi-step tasks. Gorilla [6] fine-tunes a 7 B model for API calls and pairs it with a document retriever for test-time adaptation, evaluating on APIBench. ToolLLM [7] scales this to 16k+ real APIs with depth-first-search annotation (DFSDT) and a neural API retriever. Yantra differs in two ways: routing is *hard-decoupled* from generation through an explicit bind contract (the LM never sees tool selection as its job), and the whole recipe runs free on a single T4 with no teacher-API budget. We also deliberately stay single-shot: unlike ReAct-style traces, there is no reasoning scratchpad, which keeps 1 B outputs short and parseable at the cost of no planning.

Data and benchmarks. Our training and evaluation data derive from ToolACE [1], an agentic pipeline (self-evolution synthesis, multi-agent dialog generation, dual-layer verification) producing 26,507 APIs and verified dialogs; its 8 B model rivals GPT-4-class function calling. For measurement, the Berkeley Function-Calling Leaderboard (BFCL) [8] is the de-facto standard (AST-based grading, multilingual and multi-turn tracks), and APIBank [9] covers full plan-retrieve-execute workflows. We evaluate on a ToolACE slice rather than BFCL because our harness, format, and

Yantra — DTSA inference (decoupled routing + args) + training pipeline

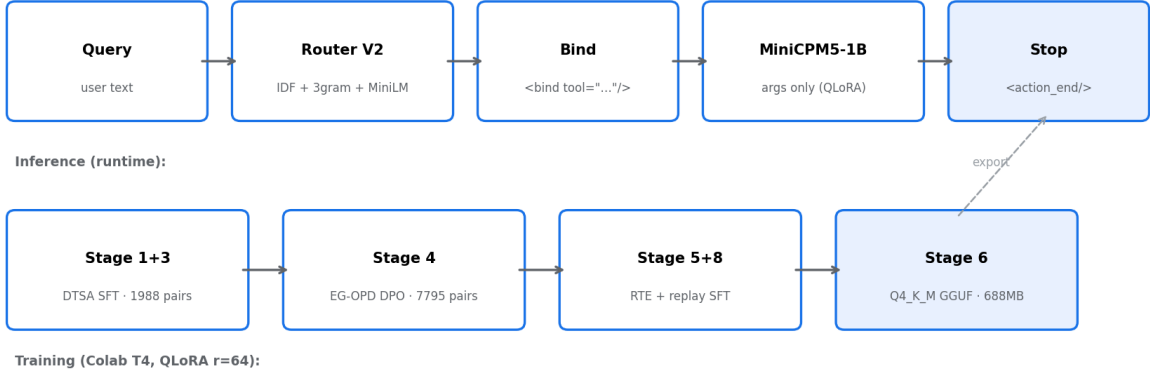


Figure 1: DTSA inference (top): the router binds the tool, the 1 B model emits arguments only, fixed stops terminate generation. Training (bottom): SFT, preference optimization, error-recovery SFT, replay, and GGUF export on a Colab T4.

single-call scope were built around ToolACE from day one; a BFCL port is explicit future work (Section 7), and we make no cross-benchmark claims.

Small models and efficient tuning. Our base, MiniCPM5-1 B [2, 3], is a 1.08 B-parameter Llama-architecture causal LM (24 layers, grouped-query attention, 131 K native context, Apache-2.0) from the MiniCPM family, whose training methodology (wind-tunnel scaling, WSD scheduling) targets strong SLMs [2]. Notably, its own post-training uses RL plus on-policy distillation, and its card documents native XML-style tool calling served via SGLang – a fact central to our baseline discussion in Section 4.3. Recent work on sub-1B models [15] and frozen-base correction [16] shows the space is moving fast. We fine-tune with QLoRA [4] ($r=64$, $\alpha=128$) and use Direct Preference Optimization [5] in one stage; Section 6.2 contributes a cautionary datapoint on DPO with tiny contrastive sets.

3 Method

Figure 1 overviews the system: a decoupled inference path (top) and the training pipeline that produces the weights (bottom).

3.1 DTSA: decoupled tool selection / argument generation

Given a user query q and a candidate tool list $T = \{t_1, \dots, t_k\}$ (JSON schemas with names and descriptions), a conventional agent prompts the LM with (q, T) and asks it to produce the complete call. DTSA splits this into two contracts. First, a deterministic router picks

$$t^* = \text{route}(q, T), \quad (1)$$

and the runtime binds it. The LM is then prompted with

```

<user>{q}</user>
<tools>{T as JSON}</tools>
<calls><bind tool="{t*}" />

```

and must complete *only* the argument block:

```

<args><param name="k1">v1</param>...</args><action_end/>

```

The split has three consequences. (1) The LM’s task collapses from open-ended tool reasoning to schema-constrained slot filling, which is within a 1 B model’s reach. (2) Routing accuracy is measurable *without any LLM call* (gold tools are known), so router improvements are cheap, exact, and decoupled from training. (3) Upgrading the router never invalidates the weights.

3.2 Router V2 with embedding blend

Let $\text{tok}(s)$ be lowercase alphanumeric tokens and $\text{tri}(s)$ character 3-grams. Over a tool registry with documents $d_i = \text{tok}(\text{name}_i + \text{description}_i)$, inverse document frequencies are $\text{idf}(w) = \log(N/(1 + \text{df}(w)))$. For query q with $Q_t = \text{tok}(q)$, $Q_g = \text{tri}(q)$, tool t scores

$$s_{\text{lex}}(q, t) = \frac{\sum_{w \in Q_t \cap d_t} \text{idf}(w)}{\sqrt{\sum_{w \in d_t} \text{idf}(w) + \epsilon}} + 2.0 \frac{|Q_g \cap \text{tri}(\text{name}_t)|}{\sqrt{|Q_g| \cdot |\text{tri}(\text{name}_t)| + 1}}. \quad (2)$$

IDF statistics are fit once over the full registry (`fit_corpus`) and reused for per-query subsets, mirroring the benchmark construction. Optionally, cosine similarities s_{emb} from `all-MiniLM-L6-v2` (tool text “name — description”) are min-max normalized per query alongside s_{lex} and blended as $(1 - \beta) s_{\text{lex}} + \beta s_{\text{emb}}$ with $\beta = 0.5$ chosen by sweep. Lexical-only routing needs zero extra downloads; the blend costs one 80 MB embedding model at runtime.

3.3 Training pipeline

Base model MiniCPM5-1 B is fine-tuned with QLoRA ($r=64$, $\alpha=128$, dropout 0.05) through the stages in Table 1. Prompts carry the router’s bind; completions are args-only, exactly matching inference. A second DPO round on 66 mined argument failures is *excluded* from the released weights: it collapsed (Section 6.2); a supervised replay step on the same fixes plus 200 replay rows was used instead. The final adapter is merged and exported to Q4_K_M GGUF (688 MB). Total compute is roughly 3 hours on a free Colab T4; no paid teacher or data-generation APIs are used at any stage.

3.4 Inference contract

Decoding uses temperature 0, $n_{\text{ctx}}=4096$ (of 131 K native), `max_tokens` 768, and six fixed stop strings (newline-bind, tool-result, user, end-calls, tool-error, reflect; Appendix B). The parser takes the *last* bind with an argument block (later binds supersede earlier ones) and falls back to tagless `name="k">v` lines, because the base tokenizer treats `<param>` as special tokens and may strip them. Generation is graded on prompt-bind + completion jointly, since the completion is args-only by construction.

Table 1: Training stages (Colab T4, QLoRA). Stage numbering follows the pipeline notebook; the collapsed DPO round is omitted from the released weights and analyzed in Section 6.2.

Stage	Method	Data	Epochs	LR	Time
1	Data builder	ToolACE extract	—	—	5 min
3	DTSA SFT	1,988 pairs	2	10^{-4}	30 min
4	EG-OPD DPO	7,795 pairs	1	5×10^{-5}	2 h
5	RTE SFT	error corrections	1	10^{-4}	15 min
8	SFT replay	66 fixes + 200 replay	1	10^{-5}	10 min
6	Export	Q4_K_M GGUF	—	—	10 min

4 Evaluation Protocol

4.1 ToolACE-300

We extract single-call ToolACE dialogs (7,822 parseable of 11,300; 4,382 single-call) and sample a deterministic 300-case set (seed 42). Each case holds a query, a candidate tool list, and one gold call. Cases are single-call and English only. The set, builder script, and seed are released so the exact slice rebuilds bit-for-bit given the upstream dataset snapshot.

4.2 PAS: Primary Aggregated Score

Let a prediction be graded on six measured components – parseable, valid tool name, expected (gold) tool name, exact argument match (keys *and* values), argument-key overlap, stopped cleanly – each in $[0, 1]$, plus recovery and multiturn, currently unsupported and fixed at 0 (Appendix A). PAS is their unweighted mean:

$$\text{PAS} = \frac{1}{8} (m_{\text{parse}} + m_{\text{valid}} + m_{\text{expect}} + m_{\text{exact}} + m_{\text{overlap}} + m_{\text{stop}} + 0 + 0). \quad (3)$$

Equal weighting is a deliberate choice: it rewards usable systems (parseable, correct tool, correct args, clean stop) without letting any single axis dominate, and the two zero components penalize the headline honestly for missing capabilities instead of hiding them.

4.3 Baseline and the serving caveat

Our reference is the public agentic 1 B baseline (MiniCPM5 agentic-tooluse GGUF, Q4_K_M)¹, served through llama.cpp with the identical harness, prompts, and stops as Yantra. It scores PAS **0.1922**. Its characteristic failure under this serving is emitting schema echoes or mangled calls (e.g., bare `name="...">` fragments) rather than the trained format. This is *expected*: the baseline targets SGLang’s native `minicpm5` stack (its model card documents a dedicated tool-call parser), which was unavailable in our environment. Its card-claimed numbers therefore cannot be reproduced through llama.cpp, and we do not try. Our claim is strictly **system-vs-system under identical serving**: Yantra beats the reference 3.52× when both are served the same way. This is the relevant comparison for practitioners who must deploy through generic inference stacks.

Because the reference outputs are parser-sensitive under generic serving (tolerant vs. strict parsing moves its sub-metrics while its headline stays poor), we report *only* its headline PAS from the controlled baseline run and omit its sub-metric row. This avoids mixing harnesses in one table – a comparison the reader could not verify.

¹Repository: <https://huggingface.co/ewinregirgojr/MiniCPM5-1B-Agentive-Tooluse-GGUF>.

Table 2: ToolACE-300 results ($n=300$). Yantra components are from the single official run (mean = 0.6775). Baseline is the headline PAS of the controlled reference run under identical serving; its sub-metrics are omitted per Section 4.3.

Metric	Baseline	Yantra
Parseable	—	1.0000
Valid tool name	—	1.0000
Expected tool name	—	0.8233
Exact args	—	0.6500
Arg key overlap	—	0.9467
Stopped cleanly	—	1.0000
PAS	0.1922	0.6775

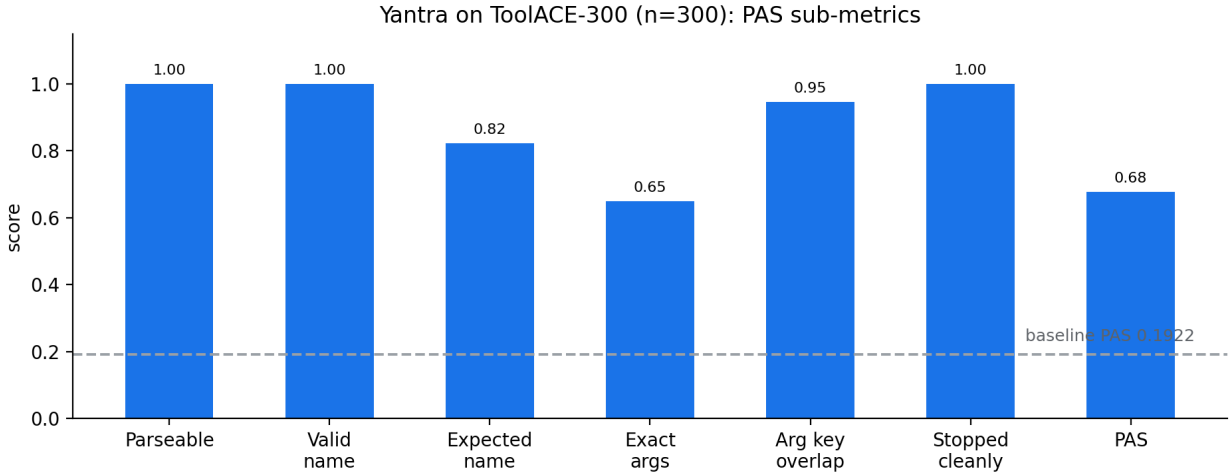


Figure 2: Yantra PAS sub-metrics on ToolACE-300. Dashed line: baseline PAS 0.1922 under identical serving. Baseline sub-metric bars are intentionally omitted (Section 4.3).

4.4 Reproducibility

All runs use temperature 0 and fixed seeds; the pipeline notebook is resumable (completed stages auto-skip after preemptions). Router benchmarks run *before* any LLM eval (gold labels known), and the final eval promotes results only if PAS improves (guarded promote). Per-case predictions for the official run are retained with the run artifacts; the repository ships the harness, seeds, and the exact notebooks.

5 Results

Table 2 and Figure 2 give the headline: PAS **0.6775** vs. **0.1922** ($0.6775/0.1922 = 3.52\times$). Every output parses; every stop is clean; argument keys are right 94.67% of the time while exact argument values remain the headroom at 65% (195/300; 95% Wilson CI 0.59–0.70). Yantra’s expected-name rate (0.8233) equals the router ceiling by construction, since the model never selects tools.

Table 3: Router benchmark on ToolACE-300 gold labels (no LLM). 95% Wilson intervals. The blend is the shipped default.

Router	Accuracy	95% CI
Lexical (IDF + char-3-gram)	244/300 (0.813)	0.77–0.85
Embedding (MiniLM-L6-v2)	242/300 (0.807)	0.76–0.85
Blend ($\beta=0.5$)	247/300 (0.823)	0.78–0.86

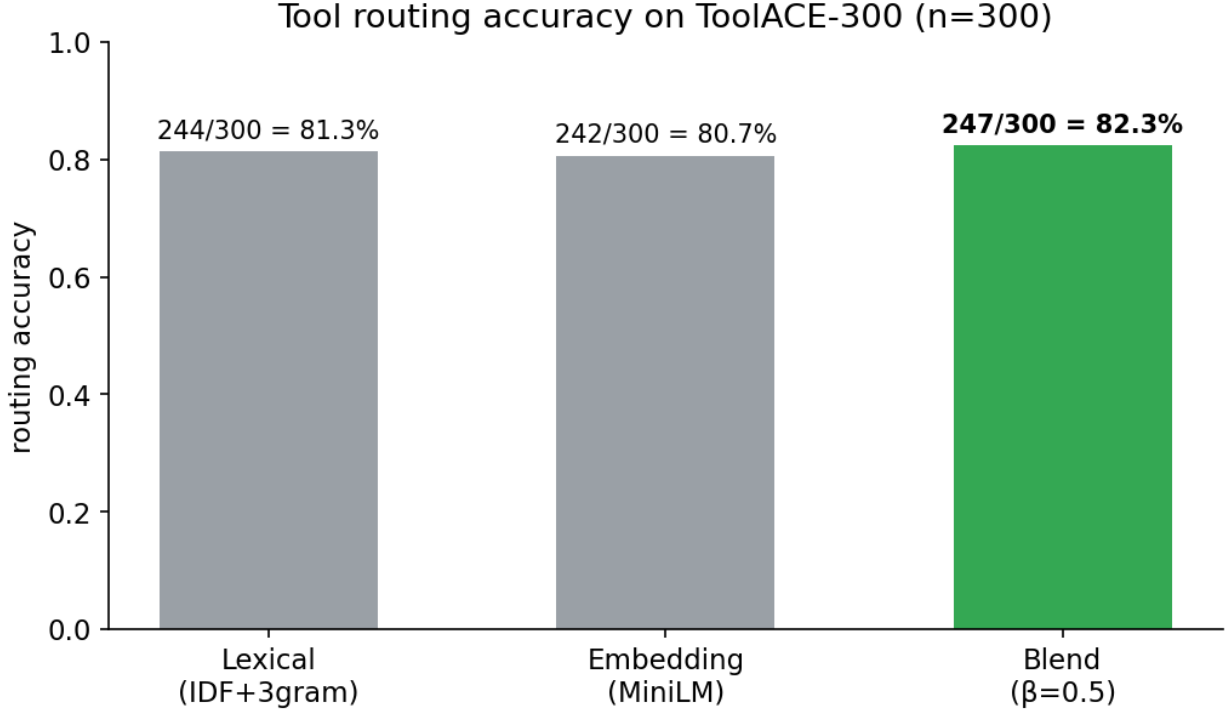


Figure 3: Routing accuracy by variant. The blend recovers three lexical misses (244 $\rightarrow$ 247/300) with no retraining.

Routing (Table 3, Figure 3) confirms the design bet: tool selection is solvable in cheap code. The intervals overlap – we do not claim the blend is significantly better, only that it is *never worse in the benchmark* and fixes qualitatively clear semantic misses (Section 6.1), at a one-time 80 MB cost.

Upper-bound analysis. With perfect routing (`expected_name` = 1.0) the PAS ceiling is $(1 + 1 + 1 + 0.65 + 0.9467 + 1)/8 = 0.70$. The exact-args (0.65) and overlap (0.9467) scores are properties of the language model alone – routing can’t improve them. Yantra’s 0.6775 already reaches 96.8% of this ceiling; the remaining gap is entirely in argument accuracy, which is the hard part of the problem and the target of future work at larger scales.

6 Analysis and Negative Results

6.1 Where the embedding blend helps

The three recovered routes are all lexical-semantic gaps, e.g., symptom-style queries mapping to drug-name tools and a “VR game for Oculus Quest” style query mapping to a `getVRGame` tool whose name shares almost no tokens with the request. Character 3-grams cannot bridge these; MiniLM cosine can. For reference, the lexical upgrade itself fixed 17 routes with 0 regressions in the standalone Router V2 study ($227 \rightarrow 244/300$); the blend adds three more ($\rightarrow 247/300$).

6.2 Small-data DPO collapses: a negative result

After mining 66 argument failures, we ran DPO for 3 epochs at 5×10^{-5} on 66 contrastive pairs. The run collapsed: `exact_args` fell $0.65 \rightarrow 0.14$ (PAS 0.5124) as the policy learned to emit a single blob parameter keyed by the tool name instead of per-key `<param>` elements. Sixty-six pairs are evidently too few for contrastive updates at this step budget – the preference gradient rewards the degenerate shortcut closest in KL to the reference on tiny data. We discarded all artifacts from this round (no checkpoint ships) and report it so others skip this configuration: *do not run few-shot DPO on structured-output models without first checking emission validity.*

6.3 Supervised replay recovers safely

Supervised fine-tuning on the same 66 fixes plus 200 replay rows (1 epoch, 10^{-5}) plateaued at PAS ≈ 0.6747 with no metric regressing: SFT-only is the safe small-data update, and replay prevents forgetting. Fixed stop sequences independently moved stopped cleanly $0.12 \rightarrow 1.00$ (PAS 0.5473 $\rightarrow 0.6746$ pre-blend). The final $+0.0029$ to 0.6775 comes from the embedding blend. Each gain is small; their sum is the system.

7 Limitations and Threats to Validity

We state these plainly so the work is judged on what it is: a 1 B-scale, single-benchmark systems recipe. (i) Single eval slice ($n=300$), single seed, greedy decoding: we report Wilson intervals for accuracies but no multi-seed variance, and no confidence interval for PAS itself (per-case scores for the official run live with the run artifacts, not the repo). (ii) Single-call English only; recovery and multiturn PAS components are 0 by construction – the headline 0.6775 already prices this in. (iii) No external-benchmark validation: BFCL and APiBank are future work, and cross-benchmark comparisons would be invalid without a format port. (iv) Baseline asymmetry (Section 4.3): the reference is strong under its native stack; our $3.52\times$ holds under identical generic serving only. A SGLang-native rematch is the obvious follow-up. (v) 65% exact args means one call in three has a wrong value: schema validation before execution is mandatory, and high-stakes tool use (payments, medical, infrastructure) is out of scope. (vi) The recipe is demonstrated at 1 B; the 7 B scaling claim is a hypothesis, not a result.

8 Conclusion

Decoupling tool selection from argument generation turns 1 B tool calling from a joint-reasoning problem into two tractable ones: routing in testable code (247/300) and args-only generation in a

small fine-tune, glued by a strict stop-terminated contract. Yantra reaches PAS 0.6775 on ToolACE-300 – $3.52\times$ an identically served reference – reproducibly on a free T4 with zero API spend. The durable lessons are arguably the small ones: fix your stops before your losses, blend cheap signals before training bigger ones, and never trust few-shot contrastive updates on structured outputs. Weights, code, and notebooks are public; the failure modes are documented next to the wins.

Reproducibility Statement

Pipeline, eval harness, seeds (42), decoding settings (temperature 0, `max_tokens` 768, stops in Section 3.4), and checkpoints are released at <https://github.com/eulogik/yantra> with weights at <https://huggingface.co/eulogik/yantra-1b-agent>. The full pipeline reruns on a free Colab T4 in about three hours; router benchmarks rerun CPU-only in minutes. Known gaps: per-case official scores are archived with run artifacts rather than the repo, and the upstream ToolACE snapshot may drift (we pin our extractor and seed; rebuild from the released slice for bit-exact comparison).

Acknowledgments

Base model MiniCPM5-1 B by OpenBMB; training and eval data from the Team-ACE ToolACE release; training via Unsloth (QLoRA), inference via llama.cpp, embeddings via sentence-transformers. We thank the authors of these open artifacts, without which a zero-budget project of this kind is impossible.

References

- [1] Team-ACE. ToolACE: Winning the points of LLM function calling. *arXiv preprint arXiv:2409.00920*, 2024.
- [2] Shengding Hu, Yuge Tu, Xu Han, et al. MiniCPM: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.
- [3] OpenBMB. MiniCPM5-1B model card. <https://huggingface.co/openbmb/MiniCPM5-1B>, 2025. 1.08 B Llama-architecture causal LM, Apache-2.0, native tool calling via SGLang.
- [4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient fine-tuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.14314.
- [5] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.18290.
- [6] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.
- [7] Yujia Qin, Shihao Liang, Yining Ye, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.16789.

- [8] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): from tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, PMLR 267, 2025.
- [9] Minghao Li, Feifan Song, Bowen Yu, et al. API-Bank: A benchmark for tool-augmented LLMs. *arXiv preprint arXiv:2304.08244*, 2023.
- [10] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [11] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [12] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [13] Unsloth AI. Unsloth: Easiest tool to fine-tune LLMs. <https://github.com/unslothai/unsloth>, 2024.
- [14] Georgi Gerganov et al. llama.cpp: LLM inference in C/C++. <https://github.com/ggerganov/llama.cpp>, 2023.
- [15] Gautam Kishore. pico-type: A 1.5M-Parameter Byte-Level Multi-Head Content Classifier. *arXiv preprint arXiv:2608.14658*, 2026.
- [16] Gautam Kishore. Safe Error Correction for Language Models: Frozen-Base Adjustment with Capability Preservation. *arXiv preprint arXiv:2609.16145*, 2026.
- [17] Gautam Kishore. Yantra: A reproducible recipe for sub-1B agentic tool calling with decoupled routing. *arXiv preprint arXiv:TODO*, 2026. Weights: <https://huggingface.co/eulogik/yantra-1b-agent>; code: <https://github.com/eulogik/yantra>. MIT license.

A PAS definition

For one case let $p \in \{0, 1\}^6$ be parseable, valid name, expected name, exact args, overlap = $|K_{\text{pred}} \cap K_{\text{gold}}|/|K_{\text{gold}}|$ (1 if gold has no args), stopped cleanly. With recovery and multiturn fixed at 0, $\text{PAS} = \frac{1}{8}(\sum p + 0 + 0)$. Dataset PAS is the mean over cases. Yantra’s official run: $(1 + 1 + 0.8233 + 0.65 + 0.9467 + 1)/8 = 0.6775$.

B Prompt, stops, and parser

The exact inference prompt (router output `t*` spliced in):

```
<user>{query}</user>
<tools>[{name, description, parameters}...]</tools>
<calls><bind tool="{t*}">
```

Stop strings (literal newline shown as `\n`):

- `"\n<bind", "<tool_result>", "<user>"`

- "</calls>", "<tool_error>", "<reflect>"

Parser: find all bind tags; scan binds last-to-first for the first with a following <args> block before the next bind; parse <param> pairs, else tagless `name="k">v` lines; stopped iff trailing text after the last action-end tag is empty.

C Hyperparameters

Setting	Value
Base	openbmb/MiniCPM5-1B (Apache-2.0)
Adapter	QLoRA $r=64$, $\alpha=128$, dropout 0.05
SFT LR / epochs	10^{-4} , 2 (DTSA); 10^{-4} , 1 (RTE)
DPO (stage 4)	5×10^{-5} , 1 epoch, 7,795 pairs
Replay SFT	10^{-5} , 1 epoch, 66 fixes + 200 replay
Decoding	temperature 0, <code>max_tokens</code> 768, n_{ctx} 4096
Router blend	$\beta = 0.5$, MiniLM-L6-v2 (80 MB)
Eval	ToolACE-300, seed 42, single-call English
Export	Q4_K_M GGUF, 688 MB

D Worked example

Query: “Find me a VR game for Oculus Quest.” Router V2 binds `getVRGame`; the model completes:

```
<args><param name="platform">Oculus Quest</param><param
  name="genre">adventure</param></args><action_end/>
```

Lexical-only routing misses this case (“VR game” shares almost no tokens with `getVRGame`); the MiniLM blend recovers it – the mechanism behind two of the three $244 \rightarrow 247$ routing fixes.